

智能体智运系统设计书

面向复杂软件系统的可插拔工具增强型智能体运维诊断与闭环处置系统

LGagent / LatentGuard

2026 年 6 月 1 日

目录

1	文档概述	3
1.1	编写目的	3
1.2	系统定位	3
2	建设背景与设计目标	3
2.1	背景	3
2.2	设计目标	3
3	总体架构设计	3
3.1	架构分层	3
3.2	总体设计思路	4
3.3	总体架构图	4
4	功能架构	4
4.1	前端模块	4
4.2	后端接口模块	5
5	智能体设计	5
5.1	核心执行框架	5
5.2	智能体原理图	6
5.3	日志诊断与安全闭环链路	6
5.4	智能体原理图	6
5.5	日志诊断与安全闭环链路	7
6	工具系统设计	7
6.1	工具分类	7
6.2	工具生成机制	8
6.3	工具扩展原则	8
6.4	工具架构图	8

7 RAG 设计	9
7.1 知识组织	9
7.2 索引与检索流程	10
7.3 自动同步机制	10
7.4 RAG 架构图	10
8 非侵入式监控设计	10
8.1 设计原则	10
8.2 采集方式	10
8.3 降噪与解耦	11
8.4 监控运维原理图	11
9 数据设计	11
9.1 mysql 数据表	11
9.2 向量数据	12
10 闭环处置流程	12
11 安全性与可扩展性考虑	12
11.1 安全性	12
11.2 可扩展性	12
12 结论	13

1 文档概述

1.1 编写目的

本文档用于说明智能体智运系统的总体设计方案、系统组成、关键流程、模块职责、数据结构及部署运行方式，为项目研发、测试、演示、维护和后续扩展提供统一依据。

1.2 系统定位

本系统面向复杂软件系统运维场景，围绕“日志感知、智能诊断、工具扩展、任务闭环、经验沉淀”构建一体化能力。系统既支持对存量系统进行低侵入接入，也支持对新业务系统进行更深层次的接口扩展和工具联动。

2 建设背景与设计目标

2.1 背景

随着企业数字化转型深入，系统形态逐渐从单体应用演进为微服务、容器化以及多节点异构部署。故障排查所需日志分散在不同服务器、不同进程和不同业务模块中，传统依赖人工巡检与经验判断的方式效率较低。

同时，传统监控与 APM 工具通常需要探针、SDK 或运行时改造，对于已经稳定运行的存量系统而言，实施成本较高，且容易引入新的不确定性。因此，需要一套既能快速接入复杂环境，又能提供智能化诊断和闭环处置能力的运维系统。

2.2 设计目标

支持对复杂异构服务环境进行统一运维诊断。

对存量系统提供低侵入甚至零侵入的日志接入能力。

将知识库、经验库、智能体推理与工具执行结合起来，提高复杂故障定位效率。

支持外部工具动态扩展，增强系统的持续适配能力。

建立从异常发现、根因诊断、任务生成到人工反馈沉淀的闭环机制。

3 总体架构设计

3.1 架构分层

系统从整体上划分为四层：

1. 用户交互层：负责提供前端页面、可视化交互、问答窗口、日志节点管理、任务与知识展示。
2. 后端 API 与服务层：负责对外提供 REST 与 SSE 接口，并承载会话、日志、知识库、工具等服务逻辑。

- 智能体核心层：负责基于 LangGraph 的 ReAct 推理、工具调用、日志诊断和可观察执行轨迹输出。
- 知识、监控与数据层：负责节点配置、SQLite 持久化、知识文档管理、向量索引与远程日志采集。

3.2 总体设计思路

系统围绕一个核心思路组织：先以日志、知识和配置等结构化输入构建可靠的上下文，再由智能体执行多步推理与工具调用，最终把输出结果落到任务卡片和经验库中，形成可持续优化的运维闭环。

3.3 总体架构图

图 1 展示了系统从用户交互层、后端服务层、智能体核心层到知识与数据层的整体结构，以及从日志感知到诊断卡片和经验沉淀的闭环路径。

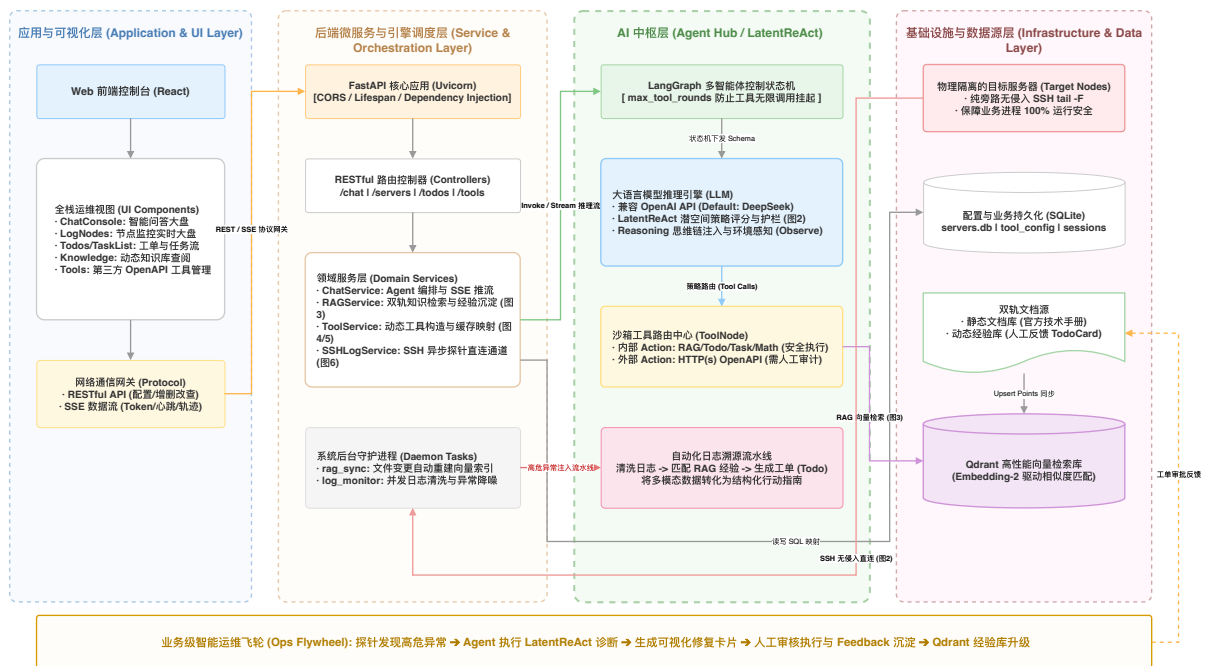


图 1: 智能体智运系统总体架构图

4 功能架构

4.1 前端模块

前端采用 React + Vite + Ant Design 实现，主要页面包括：

ChatConsole：提供智能问答、SSE 流式输出、工具轨迹可视化。

LogNodes：提供日志节点管理、实时日志展示、Agent 诊断过程展示。

Todos：展示待处理诊断卡片，支持确认修复、人工反馈。

TaskList: 管理任务清单项，支持状态更新与删除。

Knowledge: 展示知识库文档及索引状态。

Tools: 展示系统工具和外部工具，支持启停与导入。

4.2 后端接口模块

后端以 FastAPI 为核心，按职责拆分为以下接口域：

/chat: 智能体问答、会话持久化、系统提示词管理。

/servers: SSH 节点管理、日志读取、SSE 事件订阅。

/todos: 诊断卡片创建、状态流转、人工反馈。

/task-list: 任务清单增删改查。

/knowledge: 知识库文档浏览、上传、重建与检索。

/api/tools: 系统工具和外部工具管理。

5 智能体设计

5.1 核心执行框架

系统突破了传统的 ReAct 范式，采用基于 LangGraph 构建的 **LatentReAct** 智能体执行架构。在标准的“推理-行动”循环中，创新性地引入了潜空间合规验证机制。主要节点包括：

assistant: 调用大模型，进行故障根因推理并生成工具调用请求。

latent_guard: 潜空间验证节点（核心创新）。在工具被真正执行前，利用分解式潜空间验证 (Decomposed Latent Verification) 对高危操作的安全性及合规性进行逻辑博弈与评估，防止智能体“幻觉”引发二次故障。

tools: 安全验证通过后，负责执行系统工具或外部诊断工具。

record_tool_round: 记录工具调用轮数。

tool_limit: 达到轮数上限后强制中止，防止智能体陷入死循环。

执行路径从传统的 ReAct 升级为带安全护栏的 LatentReAct 闭环：

$$assistant \rightarrow latent_guard \rightarrow tools \rightarrow record_tool_round \rightarrow assistant$$

只有当 **latent_guard** 校验通过时，工具操作才会被放行；若校验失败，系统将立刻拦截并生成安全预警。

5.2 智能体原理图

图 3 展示了基于 LatentReAct 机制的智能体执行原理。图中重点突出了在调用“状态变更”类工具前，智能体是如何在内部潜空间独立完成安全对抗与指令清洗的。

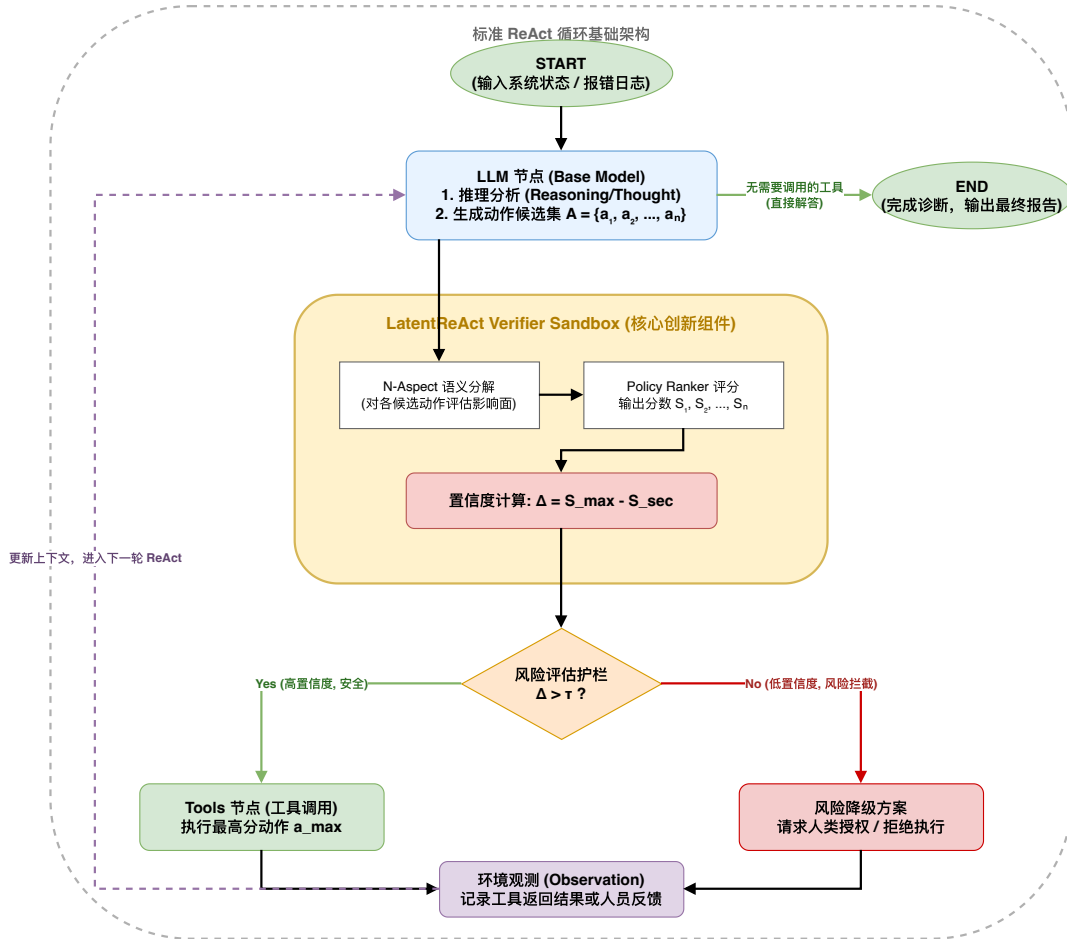


图 2: 基于 LatentReAct 机制的智能体执行原理图

5.3 日志诊断与安全闭环链路

对于实时监控场景，系统将新增错误日志提取为结构化输入，并交由智能体执行。智能体首先检索 RAG 知识库获取历史故障特征，生成初步修复方案；随后，强制触发 LatentReAct 机制对修复方案进行沙箱级预演。在确认无架构破坏与业务风险后，系统输出报错简述、触发原因与处理建议，并自动生成需人工审批的待处理卡片，完成人机协同的安全运维闭环。

5.4 智能体原理图

图 3 展示了基于 LatentReAct 机制的智能体执行原理。图中重点突出了在调用“状态变更”类工具前，智能体是如何在内部潜空间独立完成安全对抗与指令清洗的。

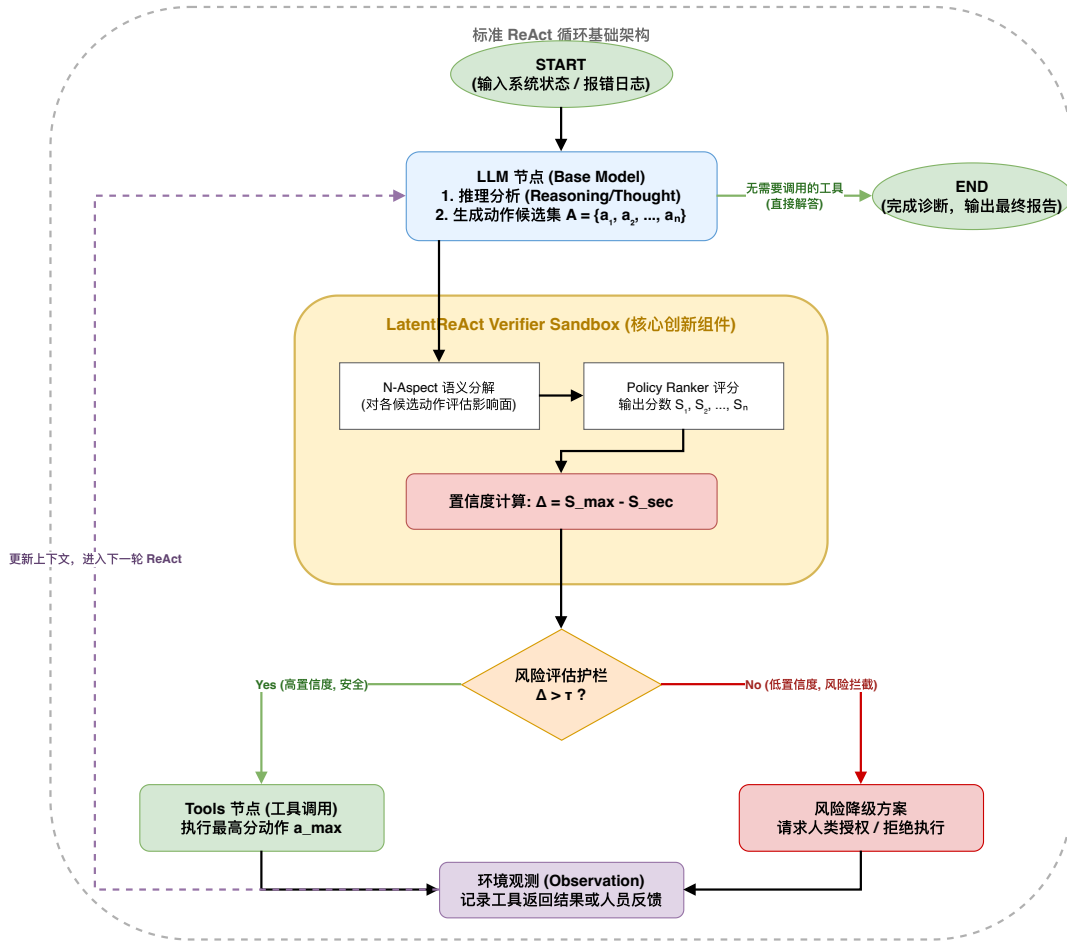


图 3: 基于 LatentReAct 机制的智能体执行原理图

5.5 日志诊断与安全闭环链路

对于实时监控场景，系统将新增错误日志提取为结构化输入，并交由智能体执行。智能体首先检索 RAG 知识库获取历史故障特征，生成初步修复方案；随后，强制触发 LatentReAct 机制对修复方案进行沙箱级预演。在确认无架构破坏与业务风险后，系统输出报错简述、触发原因与处理建议，并自动生成需人工审批的待处理卡片，完成人机协同的安全运维闭环。

6 工具系统设计

6.1 工具分类

系统工具分为两类：

系统工具：由本地 Python 函数通过 `@tool` 装饰生成，例如知识库检索、经验库检索、时间、计算、任务与卡片工具。

外部工具：由 OpenAPI 描述动态导入，运行时生成 `StructuredTool` 对象。

6.2 工具生成机制

系统工具在模块导入后已存在于内存中的 `TOOLS` 列表中。运行时, `ToolService` 根据数据库中的启停配置筛选可用工具; 外部工具则根据 `external_tool_config` 中保存的接口元数据动态构造参数模型与调用函数, 再封装为 `StructuredTool`。

6.3 工具扩展原则

新增系统工具时, 优先通过本地函数方式注册。

新增异构外部能力时, 优先通过 `OpenAPI` 导入, 减少核心代码耦合。

所有工具通过统一缓存和刷新机制进入智能体运行时。

6.4 工具架构图

图 4 展示了工具系统从系统工具、外部工具元数据, 到运行时内存生成、缓存、绑定到智能体执行图中的完整过程。

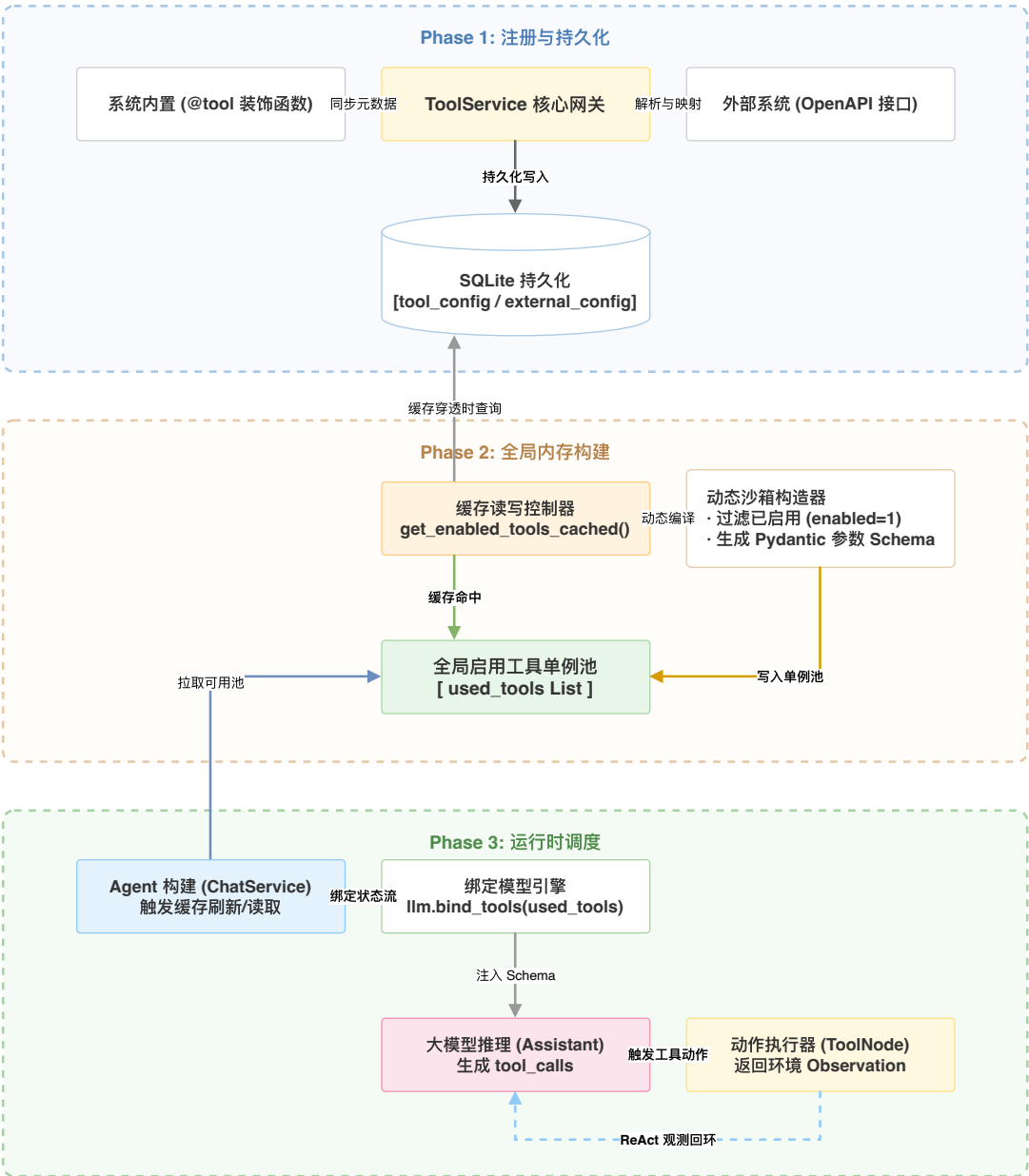


图 4: 工具调度与扩展原理图

7 RAG 设计

7.1 知识组织

系统采用双库设计：

知识库：保存通用报错知识、技术文档和排障方法。

经验库：保存人工确认过的历史处理案例。

7.2 索引与检索流程

原始文档从 `documents/` 知识库与 `documents/` 经验库读取，完成文本抽取、切分、向量化后写入 Qdrant。默认采用智谱 `embedding-2`，也支持本地与离线备用方案。查询时将用户问题或日志问题转化为向量，在对应集合中进行 Top-K 检索，并将结果返回给智能体。

7.3 自动同步机制

`rag_sync` 后台任务定期扫描文档目录变化，在检测到知识文档变更时自动触发索引重建，保证知识库内容与运行时检索结果保持一致。

7.4 RAG 架构图

图 5 展示了知识库、经验库、向量化、向量检索以及结果回流到智能体推理链路中的完整过程。

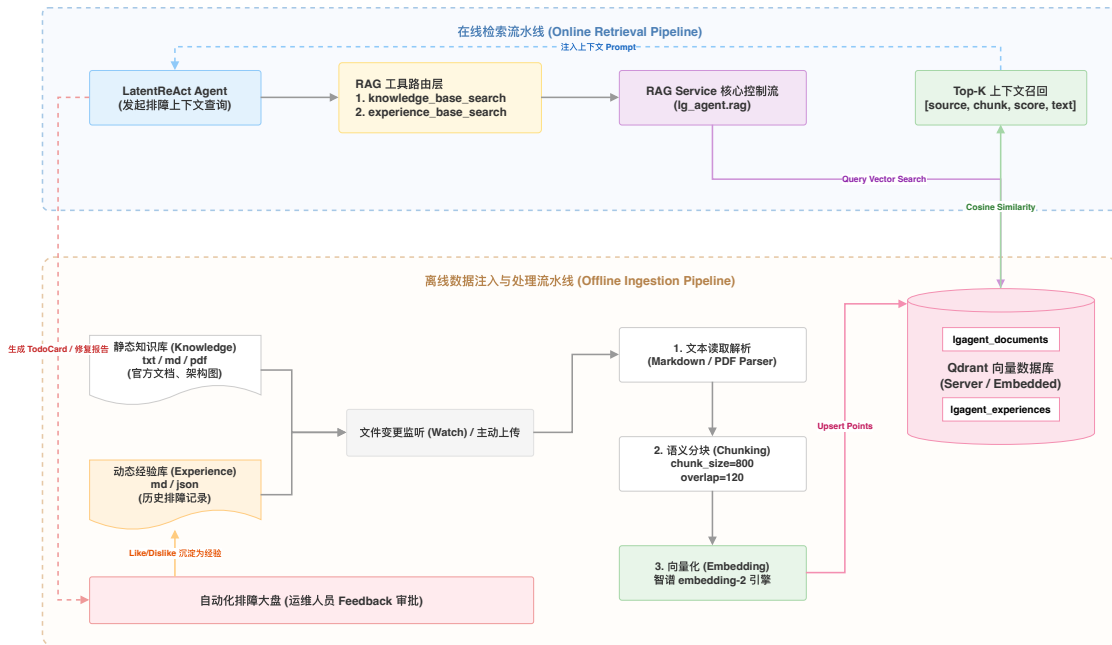


图 5: RAG 架构图

8 非侵入式监控设计

8.1 设计原则

系统对存量业务系统采用旁路式日志采集设计，目标是在不修改业务代码、不安装本地 Agent、不改变现有部署结构的前提下获取实时日志能力。

8.2 采集方式

后端通过 Paramiko 建立 SSH 连接，对远程日志文件执行 `tail -F` 读取新增内容。系统区分首次回放日志与持续追加日志，仅将新增日志中的错误块送入智能体分析链路，避免历史噪声影响。

8.3 降噪与解耦

系统对新增日志执行错误块提取和连续重复去重，并通过异步队列将采集和诊断解耦，防止模型分析耗时影响实时日志读取。

8.4 监控运维原理图

图 6 展示了旁路式日志采集、错误块提取、异步队列解耦、智能体诊断及经验反馈沉淀的全过程。

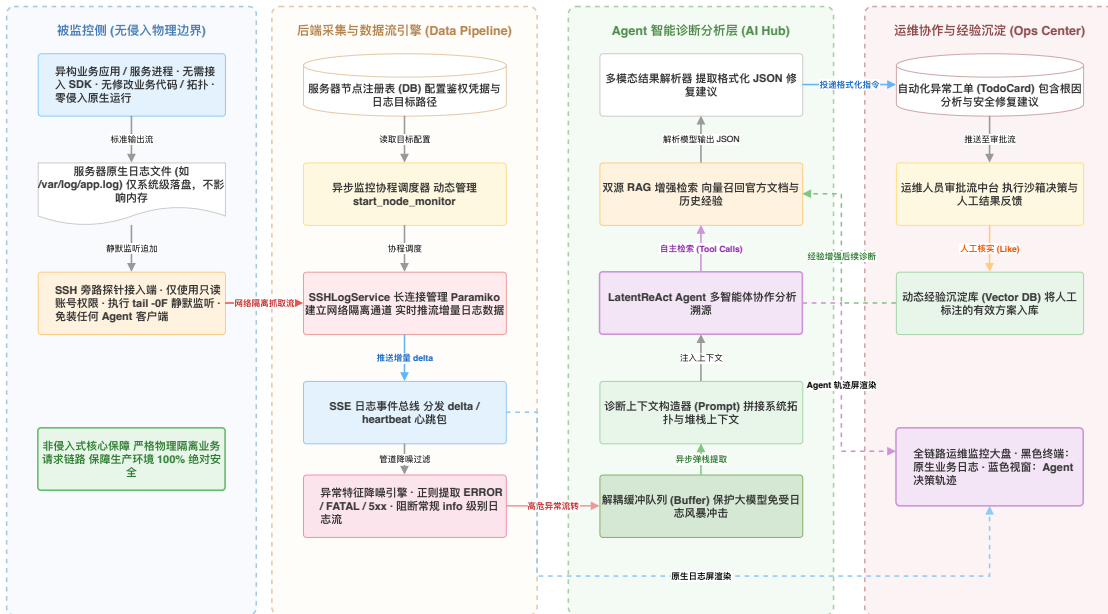


图 6: 非侵入式监控运维原理图

9 数据设计

9.1 mysql 数据表

系统当前主要使用 mysql 保存业务状态，核心表包括：

ssh_nodes: 远程节点配置。

todo_cards: 诊断卡片。

task_list: 任务清单。

tool_config: 系统工具启停配置。

external_tool_config: 外部工具元数据。

chat_sessions、chat_messages: 会话与消息持久化。

9.2 向量数据

Qdrant 中维护两个集合：

`lgagent_documents`

`lgagent_experiences`

10 闭环处置流程

系统的闭环处置流程如下：

1. 远程节点产生异常日志。
2. 后端通过旁路方式读取新增日志。
3. 系统提取错误块并去重。
4. 智能体调用知识库、经验库及工具完成诊断。
5. 系统自动生成待处理卡片。
6. 运维人员查看日志、诊断结果并进行修复。
7. 人工反馈被写入经验库。
8. 后续类似问题可直接复用经验数据。

11 安全性与可扩展性考虑

11.1 安全性

对存量系统采用只读日志采集方式，降低业务侵扰。

工具调用受统一注册与启停控制。

外部工具调用设置超时，避免长时间阻塞。

任务和日志信息均落地保存，方便审计与追踪。

11.2 可扩展性

工具层支持持续新增系统工具和 OpenAPI 外部工具。

知识库和经验库可持续扩充。

后续可对接更丰富的业务接口、处置动作和权限策略。

12 结论

智能体智运系统通过前端交互、后端服务、智能体推理、工具扩展、知识增强和闭环反馈的统一设计，实现了复杂软件系统场景下的日志感知、智能诊断与闭环处置能力。该系统兼顾工程可落地性与技术扩展性，适合作为智能运维场景下的统一研发与应用基础平台。